Computational geometry solutions

33-E01 Exact orientation and degeneracy

In (i), all four orientations are zero. Point c lies in the bounding box of ab, and the intersection is the segment from (2,0) to (6,0). In (ii), the first two orientations are -6,12, while the other two are 21,3. The latter have the same positive sign, so the segments are disjoint. Crossing the extension of ab is insufficient. In (iii), all orientations are zero, and the point c=(6,0) lies on ab, giving endpoint contact.

Coordinate differences can exceed signed 32-bit range, and products need even more bits. For a=(0,0), b=(100000,0), c=(0,100000), the orientation is $10^{10} > 2^{31} - 1$, despite modest coordinates. Widen first and check whether the global $8B^2$ bound fits the type. Full signed 32-bit inputs can require intermediates wider than signed 64 bits. The cases are overlap, separation, and endpoint intersection. Exact signs and bounds are both necessary. A fixed epsilon cannot replace an exact integer test.

33-E02 Insertions alone are insufficient

At x=0, insert a then c. The order is a,c, and they do not meet. Segment a reaches y=2 at x=4, beyond c's endpoint 3. At x=2, insert b. The order is a,c,b. Segment b reaches y=2 at $x = 22/3 > 3$, so it does not meet c. Before deleting c at x=3, a has height $3/2$ and b has height $21/4$. Test the newly adjacent a,b. Solving $x/2 = 15/2 - 3x/4$ gives x=6,y=3, inside both, so return true.

With v added, insert b first at x=2, then query the vertical interval [3,4]. Active heights are a=1,c=2,b=6. The first height at least 3 is b's 6, which exceeds 4, so v intersects no active nonvertical segment. There is no other same-x vertical interval. The detector still returns true at x=3 for a,b.

A vertical segment needs a range query over its full interval. Neither an arbitrary neighbor nor midpoint key 3.5 is sufficient. Insertions alone miss the adjacency revealed by deleting c.

33-E03 A detector is not an enumerator

Take $a : (0,0) \to (4,4), b : (0,4) \to (4,0), c : (0,5) \to (4,5)$. Before x=2, height order is a,b,c. After a,b meet at (2,2), it becomes b,a,c. Printing the intersection without exchanging status order leaves the tree in a,b,c order, so later `above` and `below` no longer describe geometric neighbors.

Computational geometry solutions (continued)

Enumeration needs predicted neighbor-intersection events in the queue, processing at their intersection time, exchanging or reordering the intersecting group, and updating new neighbor events. It also needs duplicate suppression, rejection of past events, and rules for concurrent intersections, vertical segments, and overlap. The endpoint-only proof and cost cannot be transferred unchanged.

Early termination is a premise of the detector's status proof. This example locates invariant failure after x=2. It does not claim every such three-segment example immediately misses an additional intersection.

33-E04 Hull and boundary conventions

Start at O. G shares A's ray, so keep farther A. F shares C's ray, so keep farther C. The angle order is O,A,B,C,D,E. Initially $\operatorname{orient}(O, A, B) = 6$. Adding C gives $\operatorname{orient}(A, B, C) = -8$, so pop B. Then $\operatorname{orient}(O, A, C) = 24 > 0$, leaving O,A,C. Adding D gives $\operatorname{orient}(A, C, D) = 16$, so retain it. Adding E gives $\operatorname{orient}(C, D, E) = -6$, so pop D. Then $\operatorname{orient}(A, C, E) = 24$, giving O,A,C,E.

The hull is that rectangle, area 24. B,D,F are interior and G is inside the lower edge. They are contained in the hull but omitted from the extreme-vertex output.

For all-collinear input such as (0,0),(1,1),(2,2),(1,1), deduplicate and return only (0,0), (2,2). If only one unique coordinate remains, return one point.

33-E05 Algebraic proof of popping

Since $D > 0$, r,u are linearly independent and uniquely express $t = \alpha r + \beta u$. Cross products give

$$\alpha = \frac{\operatorname{orient}(O, t, u)}{D} \geq 0, \qquad \beta = \frac{\operatorname{orient}(O, r, t)}{D} \geq 0.$$

Nonnegativity follows from t's direction between the two rays in a span below π. Expanding,

$$(t - r) \times (u - r) = ((\alpha - 1)r + \beta u) \times (u - r) = (\alpha - 1)D + \beta D = (\alpha + \beta - 1)D.$$

A nonpositive orientation therefore gives $\alpha + \beta \leq 1$. Point t is a convex combination of O,r,u with nonnegative coefficients summing to 1. Distinct from those three points, it is not extreme in the four-point hull. Equality puts it on ru. Strict inequality puts it inside the triangle or on an O-adjacent edge. In either case deleting it preserves the hull.

Angle order and a total span below π are essential for nonnegative coefficients. A nonleft turn alone for arbitrary directions does not prove triangle containment. Graham's pivot and angle order supply these conditions.

Computational geometry solutions (continued)

33-E06 Closest pairs with equal x-coordinates

Rank splitting assigns IDs 1-4 to L, with y-values 0,10,20,30 and minimum distance 10. IDs 5-8 form R, with 31,40,50,60 and minimum distance 9, between 31 and 40. Thus $\delta = 9, \ell = 0$, and every point lies in the strip. The y-order is unchanged. Cross-group IDs 4 and 5 have distance 1, the global minimum.

Using x<0 gives an empty left group and all eight points on the right, inconsistent with the rank partition. If recursion also uses that split, input size does not decrease and the balanced recurrence fails. Equal-x points need group membership by unique ID.

Packing requires only within-group distances at least δ, left-group x at most ℓ, and right-group x at least ℓ. Equality is allowed. Points on the line are counted separately by group. Strict geometric separation is unnecessary.

33-E07 Arithmetic and bit costs

Brute force checks $n(n-1)/2$ distances, or $\Theta(n^2)$. Sorting Y at each recursive level gives $T(n) = 2T(n/2) + \Theta(n \log n)$, with corresponding sorting cost $\Theta(n \log^2 n)$. Presorting and stable partitioning need only linear work per level and a constant number of successor checks per point, giving $O(n \log n)$.

Square root is strictly increasing on nonnegative values, so comparing $dx^2 + dy^2$ is equivalent to comparing distance. Integer squared distances and orientations remain exact. Differences of b-bit coordinates use about b+1 bits, and squared or cross products about 2b bits. Multiplication has cost $M(b)$, and comparison $O(b)$. Arithmetic operation bounds therefore need bit-cost factors for arbitrary precision.

Divide and conquer reduces candidate pairs and repeated sorts. Squared distance avoids unnecessary roots and their errors. Neither removes the cost of large integers.

Computational geometry solutions (continued)

Verification code appendix: geometry_lab.py

```
"""Exact integer/rational models; no floating epsilon is used for topology."""
from functools import cmp_to_key
from fractions import Fraction

def orient(a,b,c):return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
def distance2(a,b):return (a[0]-b[0])**2+(a[1]-b[1])**2
def on(a,b,p):return orient(a,b,p)==0 and min(a[0],b[0])<=p[0]<=max(a[0],b[0]) and
min(a[1],b[1])<=p[1]<=max(a[1],b[1])
def intersect(a,b,c,d):
    o1,o2,o3,o4=orient(a,b,c),orient(a,b,d),orient(c,d,a),orient(c,d,b)
    sign=lambda n:(n>0)-(n<0)
    return (sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0) or on(a,b,c) or on(a,b,d) or
on(c,d,a) or on(c,d,b)

def graham(points):
    points=sorted(set(points))
    if len(points)<3:return points,[]
    p0=min(points,key=lambda p:(p[1],p[0]))
    def compare(a,b):
        o=orient(p0,a,b)
        if o:return -1 if o>0 else 1
        return (distance2(p0,a)>distance2(p0,b))-(distance2(p0,a)<distance2(p0,b))
    candidates=sorted([p for p in points if p!=p0],key=cmp_to_key(compare));rays=[]
    for p in candidates:
        if rays and orient(p0,rays[-1],p)==0:rays[-1]=p
        else:rays.append(p)
    stack=[];trace=[]
    for p in [p0]+rays:
        popped=[]
        while len(stack)>1 and orient(stack[-2],stack[-1],p)
<=0:popped.append((stack[-1],orient(stack[-2],stack[-1],p)));stack.pop()
        stack.append(p);trace.append((p,popped,list(stack)))
    return stack,trace

def jarvis(points):
    points=sorted(set(points))
    if len(points)<2:return points
    start=min(points);p=start;hull=[]
```

Computational geometry solutions (continued)

```
    while True:
        hull.append(p);q=next(x for x in points if x!=p)
        for r in points:
            o=orient(p,q,r)
            if o<0 or (o==0 and distance2(p,r)>distance2(p,q)):q=r
        p=q
        if p==start:return hull

def closest(points):
    # IDs distinguish coincident coordinates; rank marks give exact stable splits.
    x=sorted([(p[0],p[1],i) for i,p in enumerate(points)])
    if len(x)<2:return None
    rank={p[2]:i for i,p in enumerate(x)}
    y=sorted(x,key=lambda p:(p[1],p[0],p[2]))
    def solve(x,y):
        if len(x)<=3:return min((distance2(a,b),(a,b)) for i,a in enumerate(x) for b in
x[i+1:])
        middle=len(x)//2;xl,xr=x[:middle],x[middle:];cut=rank[xr[0][2]]
        yl=[p for p in y if rank[p[2]]<cut];yr=[p for p in y if rank[p[2]]>=cut]
        best=min(solve(xl,yl),solve(xr,yr));delta2=best[0]
        if delta2==0:return best
        ell=Fraction(xl[-1][0]+xr[0][0],2)
        z=[p for p in y if (p[0]-ell)**2<delta2]
        for i,a in enumerate(z):
            for b in z[i+1:i+8]:
                if (b[1]-a[1])**2>=best[0]:break
                best=min(best,(distance2(a,b),(a,b)))
        return best
    return solve(x,y)

if __name__=='__main__':
    print('hull',graham([(0,0),(4,0),(3,1),(4,4),(1,3),(0,4),(1,1)]))
    print('closest',closest([(0,0),(1,4),(2,1),(3,7),(4,1),(5,5),(6,0),(7,8)]))
```

Verification code appendix: orientation_test.py

```
"""Use integer cross products to classify turns."""

def cross(origin, a, b):
    return (a[0] - origin[0]) * (b[1] - origin[1]) - (a[1] - origin[1]) * (b[0] - origin[0])


assert cross((0, 0), (2, 0), (1, 1)) > 0
assert cross((0, 0), (1, 2), (2, 2)) < 0
assert cross((0, 0), (1, 1), (2, 2)) == 0
print("left, right, collinear verified")
```

Computational geometry solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study